Socket programming in python

computer network

- A computer network is a system that connects numerous independent computers in order to share information (data) and resources. The integration of computers and other different devices allows users to communicate more easily.

- A computer network is a collection of two or more computer systems that are linked together. A network connection can be established using either cable or wireless media. Hardware and software are used to connect computers and tools in any network.

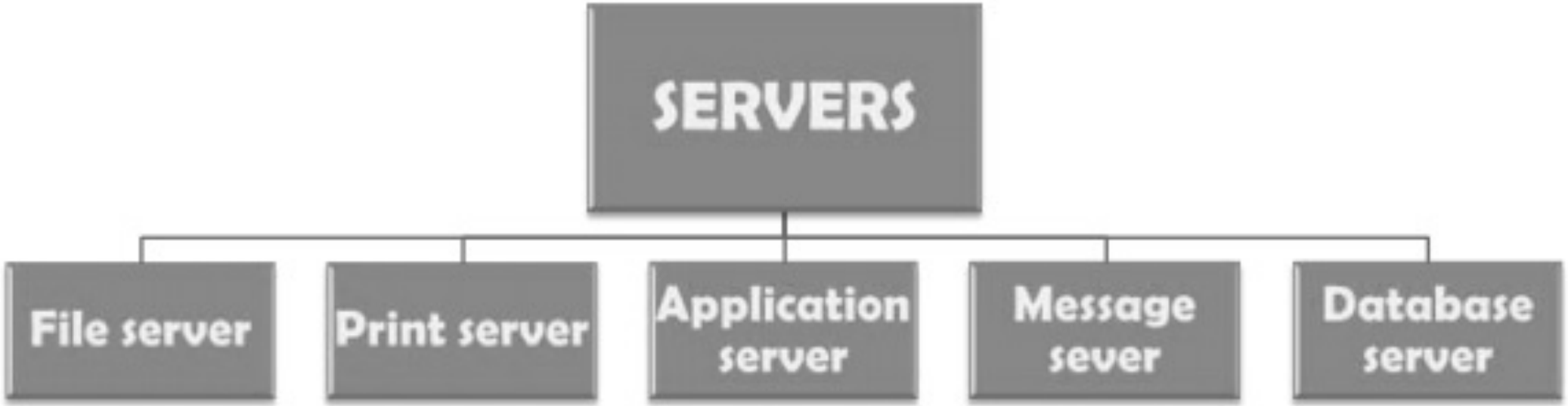
- A computer network consists of various kinds of nodes. Servers, networking hardware, personal computers, and other specialized or general-purpose hosts can all be nodes in a computer network. Hostnames and network addresses are used to identify them.

Client /server

- The computer that receive service is called client computer.
- The computer that provide service in network is called server

A server is simply a computer that provides the network resources and provides service to other computers when they request it. A client is the computer running a program that requests the service from a server. Local area network (LAN) is based on client server network relationship.

Types of Servers



Benefits of a client-server network

- Generally more secure than peer-to-peer networks
- One client computer crashing does not effect the other computers
- Easier to recover files as backups can be controlled centrally by the network administrator
- Files and resources are easier to share and control from server
- Improved levels of security as files are centralised
- It’s easier to administrate the whole network using a server
- Faster performance as each computer is only fulfilling one role
- Security is potentially cheaper and easier when done centrally
- Individual users do not have to worry about backups or security
- Larger networks can be created

Drawbacks of a client-server network

- Servers can be expensive to buy and maintain
- A network technician will often be required
- Trickier to set up with specialist knowledge needed
- Over-all set up cost is more expensive than a peer-to-peer network
- Server failure will probably disrupt all computers on the network

Protocol

- a protocol is a set of rules for formatting and processing data. Network protocols are like a common language for computers.
- Protocol is useful to physically move data from one place to another on a network, while sending or receiving data.

TCP

- TCP stands for **Transmission Control Protocol**.
- It is a transport layer protocol that facilitates the transmission of packets from source to destination.
- It is a connection-oriented protocol that means it establishes the connection prior to the communication that occurs between the computing devices in a network.
- This protocol is used with an IP protocol, so together, they are referred to as a TCP/IP
- **Reliable**
- **Order of the data is maintained**
- **Connection-oriented**
- **Full duplex**
- **Stream-oriented**

UDP Protocol

- UDP stands for User Datagram Protocol.
- UDP is the simplest transport layer communication protocol. It contains a minimum amount of communication mechanisms. It is considered an unreliable protocol
- The UDP is a connectionless protocol as it does not create a virtual path to transfer the data. It does not use the virtual path, so packets are sent in different paths between the sender and the receiver, which leads to the loss of packets or received out of order.
- **Ordered delivery of data is not guaranteed.**
- The UDP protocol uses different port numbers so that the data can be sent to the correct destination. The port numbers are defined between 0 and 1023.
- **Faster transmission**

Feature	TCP	UDP
Connection status	Requires an established connection to transmit data (connection should be closed once transmission is complete)	Connectionless protocol with no requirements for opening, maintaining, or terminating a connection
Data sequencing	Able to sequence	Unable to sequence
Guaranteed delivery	Can guarantee delivery of data to the destination router	Cannot guarantee delivery of data to the destination
Retransmission of data	Retransmission of lost packets is possible	No retransmission of lost packets
Error checking	Extensive error checking and acknowledgment of data	Basic error checking mechanism using checksums
Method of transfer	Data is read as a byte stream; messages are transmitted to segment boundaries	UDP packets with defined boundaries; sent individually and checked for integrity on arrival
Speed	Slower than UDP	Faster than TCP
Broadcasting	Does not support Broadcasting	Does support Broadcasting
Optimal use	Used by HTTPS, HTTP, SMTP, POP, FTP, etc	Video conferencing, streaming, DNS, VoIP, etc

IP address

- An IP address is a unique address that identifies a device on the internet or a local network. IP stands for "Internet Protocol," which is the set of rules governing the format of data sent via the internet or local network.
- An IP address is a string of numbers separated by periods. IP addresses are expressed as a set of four numbers — an example address might be 192.158.1.38. Each number in the set can range from 0 to 255. So, the full IP addressing range goes from 0.0.0.0 to 255.255.255.255
- Types of IP address
  - IPv4
  - IPv6

Classification of IP Address

1 Public IP Address: This address is available publicly and it is assigned by your network provider to your router, which further divides it to your devices. Public IP Addresses are of two types,

- **Dynamic IP Address:**
- **Static IP Address:**

2 Private IP Address:

3. Shared IP addresses:

4. Dedicated IP addresses:

Finding IP address varies by platform:

- **In Windows:**
- Use the command prompt.
- Search for “cmd” (without the quotes) using Windows search
- In the resulting pop-up box, type “ipconfig” (no quote marks) to find the information.
- **In Android**
- Go to **Settings**, scroll down to select **About phone**, or **System** >> **About phone**, then select **Status**.

Print IP address

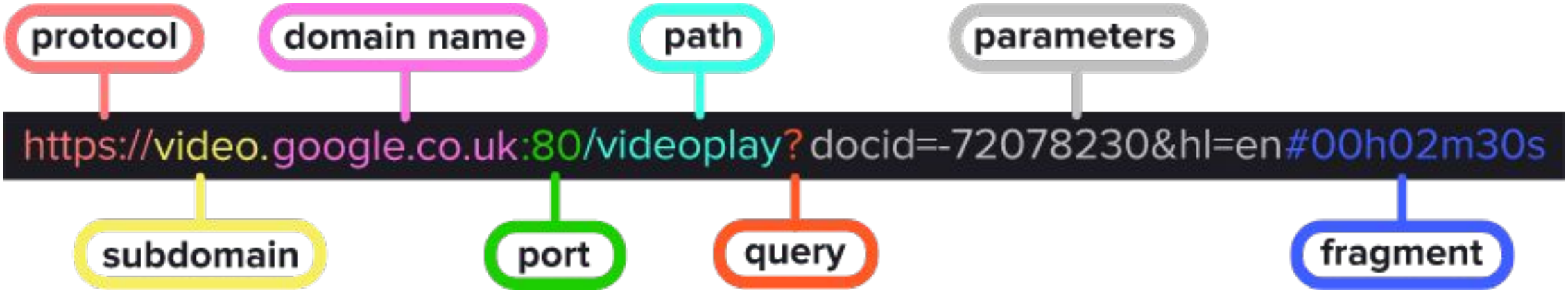
```python
import socket
hostname = socket.gethostname()
ip_address = socket.gethostbyname(hostname)
print(f"Hostname: {hostname}")
print(f"IP Address: {ip_address}")

ip_address =
socket.gethostbyname('www.gmail.com')
print(f"IP Address: {ip_address}")
```

```
Hostname: DESKTOP-B5IMBQ5
IP Address: 172.26.80.1
IP Address: 216.58.203.37
```

URL (Uniform Resource Locator)

- A URL (Uniform Resource Locator) is a unique identifier used to locate a resource on the Internet....
- it is the address of a resource, which can be a specific webpage or a file, on the internet. It is also *known as web address* when it is used with http.
- URL contains the following information:
  - The scheme name or protocol.
  - A colon, two slashes.
  - A host, normally called a domain name but sometimes as a literal IP address.
  - A colon followed by a port number.
  - Full path of the resource.

  e. g.
  - https://developer.mozilla.org
  - https://developer.mozilla.org/en-US/docs/Learn/
  - https://developer.mozilla.org/en-US/search?q=URL


protocol
domain name
path
parameters
https://video.google.co.uk:80/videoplay?docid=-720782 30&hl=en#00h02m30s
subdomain
port
query
fragment

Attribute	Index	Value	Value if not present
scheme	0	URL scheme specifier	*scheme* parameter
netloc	1	Network location part	empty string
path	2	Hierarchical path	empty string
params	3	Parameters for last path element	empty string
query	4	Query component	empty string
fragment	5	Fragment identifier	empty string
username		User name	None
password		Password	None
hostname		Host name (lower case)	None
port		Port number as integer, if present	None

```python
import urllib.parse
url='https://docs.python.org:80/3/library/urllib.parse.html
'


part=urllib.parse.urlparse(url)
print(part)
print('scheme  ' ,part.scheme)
print('netloc  ',part.netloc)
print('path  ',part.path)
print('port  ',part.port)
```

```
ParseResult(scheme='https', netloc='docs.python.org:80', path='/3/library/urllib.parse.html', params='', query='',
fragment='')
scheme   https
netloc   docs.python.org:80
path   /3/library/urllib.parse.html
port   80
```

```python
import urllib.request
file=urllib.request.urlopen('https://www.tutorialspoint
.com/python/python_networking.htm')
print(file.read())
```

port

- A port is a virtual point where network connections start and end.
- Ports are software-based and managed by a computer's operating system.
- Each port is associated with a specific process or service.
- There are 65,535 possible port numbers, although not all are in common use. Some of the most commonly used ports, along with their associated networking protocol, are:
- **Ports 20 and 21:** File Transfer Protocol (FTP). FTP is for transferring files between a client and a server.
- Port 25: Historically, Simple Mail Transfer Protocol (SMTP). SMTP is used for email.
- **Port 53:** Domain Name System (DNS). DNS is an essential process for the modern Internet; it matches human-readable domain names to machine-readable IP addresses, enabling users to load websites and applications without memorizing a long list of IP addresses.
- **Port 80:** Hypertext Transfer Protocol (HTTP). HTTP is the protocol that makes the World Wide Web possible.
- **Port 179:** Border Gateway Protocol (BGP). BGP is essential for establishing efficient routes between the large networks that make up the Internet (these large networks are called autonomous systems). Autonomous systems use BGP to broadcast which IP addresses they control.
- **Port 443:** HTTP Secure (HTTPS). HTTPS is the secure and encrypted version of HTTP. All HTTPS web traffic goes to port 443. Network services that use HTTPS for encryption, such as DNS over HTTPS, also connect at this port.
- **Port 3389:** Remote Desktop Protocol (RDP). RDP enables users to remotely connect to their desktop computers from another device.

Socket

- Sockets are the endpoints of a two-directional, point-to-point communications channel.
- Sockets may communicate within a process, between processes on the same machine, or between processes on different continents.
- Sockets are generally used to transfer the message across the network.
- For example - client (web-browser) and server . Now, there is two endpoints **Client Socket** and a **Server Socket.**
- Sockets were initially used in 1971 and later it turned into an API in the Berkeley Software Distribution (BSD) operating system which was introduced in the 1983 called Berkeley socket.

Domain	The family of protocols that is used as the transport mechanism. These values are constants such as AF_INET, PF_INET, PF_UNIX, PF_X25, and so on.
type	The type of communications between the two endpoints, typically SOCK_STREAM for connection-oriented protocols and SOCK_DGRAM for connectionless protocols.
protocol	Typically zero, this may be used to identify a variant of a protocol within a domain and type.
hostname	The identifier of a network interface – •A string, which can be a host name, a dotted-quad address, or an IPV6 address in colon (and possibly dot) notation •A string "<broadcast>", which specifies an INADDR_BROADCAST address. •A zero-length string, which specifies INADDR_ANY, or •An Integer, interpreted as a binary address in host byte order.
port	Each server listens for clients calling on one or more ports. A port may be a Fixnum port number, a string containing a port number, or the name of a service.

To create a socket, you must use the *socket.socket()* function available in *socket* module, which has the general syntax −

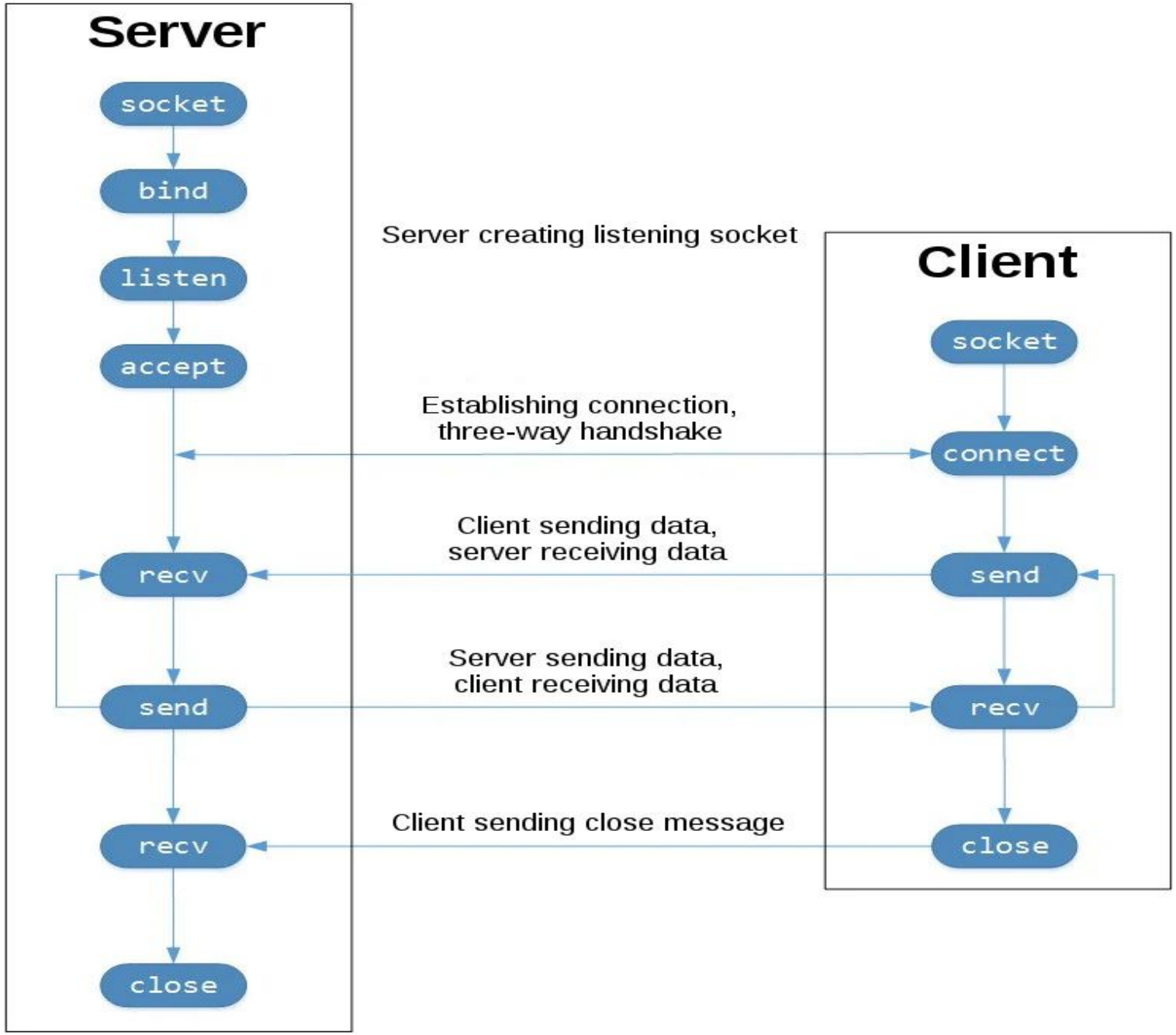
s = socket.socket (socket_family, socket_type, protocol=0)

Here is the description of the parameters −

- **socket_family** − This is either AF_UNIX or AF_INET, as explained earlier.
- **socket_type** − This is either **SOCK_STREAM** or **SOCK_DGRAM**.
- **protocol** − This is usually left out, defaulting to 0.

SOCK_STREAM used for tcp

SOCK_DGRAM. Used for udp


Server
socket
bind
listen
accept
recv
send
recv
close
Server creating listening socket
Establishing connection,
three-way handshake
Client sending data,
server receiving data
Server sending data,
client receiving data
Client sending close message
Client
socket
connect
send
recv
close

Server Socket Methods

Sr.No.	Method & Description
1	**s.bind()** This method binds address (hostname, port number pair) to socket.
2	**s.listen()** This method sets up and start TCP listener.
3	**s.accept()** This passively accept TCP client connection, waiting until connection arrives (blocking).

Client Socket Methods

Sr.No.	Method & Description
1	**s.connect()** This method actively initiates TCP server connection.

General Socket Methods

Sr.No.	Method & Description
1	**s.recv()** This method receives TCP message
2	**s.send()** This method transmits TCP message
3	**s.recvfrom()** This method receives UDP message
4	**s.sendto()** This method transmits UDP message
5	**s.close()** This method closes socket
6	**socket.gethostname()** Returns the hostname.

Client socket

```
import socket
host = '127.0.0.1'
port = 5001
obj = socket.socket()
obj.connect((host,port))
message = input("type message: ")
while message != 'q':
        obj.send(message.encode())
        data = obj.recv(1024).decode()
        print (' Received from server: ' + data)
        message = input("type message: ")
obj.close()
```

Server socket

```
import socket
host = "127.0.0.1"
port = 5001
server = socket.socket()
server.bind((host,port))
server.listen()
conn, addr = server.accept()
print ("Connection from: " + str(addr))
while True:
       data = conn.recv(1024).decode()
       if not data:
           break
       print ("from connected  user: " + str(data))
       data = str(data).upper()
       print ("Received from User: " + str(data))
       data = input("type message: ")
       conn.send(data.encode())
conn.close()
```